

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Модели решения задач в интеллектуальных системах»
Тема: «Многослойный перцептрон для XOR с ReLU на скрытых слоях: сравнение
активационных функций»

Выполнила:
Студент 4 курса
Группы ИИ-26
Карпович К.А.
Проверил:
Никитин С.К.

Цель работы: на основе многослойного персептрона архитектуры 2-2-1 из ЛР №2, исследовать влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провести сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 с функцией потерь BCE из ЛР №2.
2. Реализовать активационную функцию ReLU на скрытом слое и осуществить корректное обратное распространение ошибки для ReLU. На выходном слое оставить Сигмоиду.
3. Обучить сеть на полной таблице истинности в двух конфигурациях:
 - **Конфигурация А (базовая):** MLP 2-2-1, Сигмоида на всех слоях, BCE, нормализованные целевые значения (0/1);
 - **Конфигурация В (новая):** MLP 2-2-1, ReLU на скрытом слое, Сигмоида на выходном, BCE, нормализованные (0/1).
4. Критерий остановки для обеих конфигураций: суммарная ошибка $E_s \leq E_e$ (как в ЛР №2).
5. Для каждой конфигурации зафиксировать:
 - Количество эпох до критерия остановки.
 - Итоговую суммарную ошибку и accuracy.
 - Среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
6. Провести не менее 5 запусков обучения для каждой конфигурации с разными seed и сравнить устойчивость.
7. Построить графики:
 - График сходимости (суммарная ошибка E_s от эпохи) для обеих конфигураций на одних осях.
 - Диаграмма разброса числа эпох по 5 запускам (столбики) для конфигурации А и В.
8. Визуализация разделяющей поверхности:
 - Для представительного экземпляра каждой конфигурации построить карту (heatmap) выходного значения сети на плоскости $(A, B) \in [-10; 10]^2$ с наложением 4 точек обучающей выборки.
 - Описать, как ReLU изменяет форму разделяющей поверхности (кусочно-линейные границы vs гладкие).
9. Реализовать режим функционирования сети:
 - Пользователь вводит (A, B) из $[-10; 10]$.
 - Сеть выводит: выход в нормализованной шкале, выход в исходной шкале $[c_0; c_1]$, класс.
 - Продемонстрировать на 4 обучающих примерах + 2–3 дополнительных парах.
10. Создать таблицу сравнения (статистика по 5 запускам, MAE для обеих конфигураций).
11. Вывод: ответить на все вопросы и провести анализ результатов.

Требования к реализации

- Активационные функции: Сигмоида и ReLU.
- Веса инициализируются случайными малыми значениями.
- Обучение: метод обратного распространения в онлайн-режиме.
- Критерий остановки: $E_s \leq E_e$ или максимум эпох.
- Допускается: NumPy, Matplotlib.
- Запрещено: scikit-learn, PyTorch, TensorFlow и т.п.

Вариант 6 ($c_0 = -9, c_1 = 6$)

A	B	$A \oplus B$
-9	-9	-9
-9	6	6
6	-9	6
6	6	-9

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Patch

C0 = -9.0
C1 = 6.0
TARGET_ERROR = 0.05
MAX_EPOCHS = 10000
LEARNING_RATE = 0.5
SEEDS = [10, 42, 123, 777, 2024]
EXTRA_SEEDS = 100

X_raw = np.array([[C0, C0], [C0, C1], [C1, C0], [C1, C1]])
y_raw = np.array([C0, C1, C1, C0]).reshape(-1, 1)

def normalize(val):
    return (val - C0) / (C1 - C0)

def denormalize(val):
    return val * (C1 - C0) + C0

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -250, 250)))

def sigmoid_derivative(s):
    return s * (1.0 - s)

def relu(x):
    return np.maximum(0.0, x)

def relu_derivative(z):
    return (z > 0).astype(float)

X_norm = normalize(X_raw)
y_norm = normalize(y_raw)

class MLP:
    def __init__(self, hidden_activation, lr, seed=None):
        if seed is not None:
            np.random.seed(seed)
        self.hidden_activation = hidden_activation
        self.lr = lr

        if hidden_activation == 'relu':
            self.W1 = np.random.randn(2, 2) * np.sqrt(2.0 / 2)
            self.b1 = np.ones((1, 2)) * 1.0
            self.W2 = np.random.randn(2, 1) * np.sqrt(2.0 / 2)
            self.b2 = np.zeros((1, 1))
        else:
            self.W1 = np.random.randn(2, 2) * 0.5
            self.b1 = np.random.randn(1, 2) * 0.5
            self.W2 = np.random.randn(2, 1) * 0.5
            self.b2 = np.random.randn(1, 1) * 0.5

    def forward(self, x):
        self.z1 = np.dot(x, self.W1) + self.b1
        if self.hidden_activation == 'relu':
            self.a1 = relu(self.z1)
        else:
            self.a1 = sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2

    def hidden_derivative(self):
        if self.hidden_activation == 'relu':
            return relu_derivative(self.z1)
        return sigmoid_derivative(self.a1)

    def train_step(self, x, y_true):
        pred = self.forward(x)
        pred_clipped = np.clip(pred, 1e-15, 1 - 1e-15)
        error = y_true - pred
        delta2 = error
        delta1 = np.dot(delta2, self.W2.T) * self.hidden_derivative()
```

```

        self.W2 += self.lr * np.dot(self.a1.T, delta2)
        self.b2 += self.lr * delta2
        self.W1 += self.lr * np.dot(x.T, delta1)
        self.b1 += self.lr * delta1
        bce = -(y_true * np.log(pred_clipped) + (1 - y_true) * np.log(1 - pred_clipped))
        return np.sum(bce)

def train_model(hidden_activation, seed):
    model = MLP(hidden_activation, LEARNING_RATE, seed)
    history = []
    for epoch in range(MAX_EPOCHS):
        indices = np.arange(len(X_norm))
        np.random.shuffle(indices)
        epoch_error = 0.0
        for i in indices:
            epoch_error += model.train_step(X_norm[i:i + 1], y_norm[i:i + 1])
        history.append(epoch_error)
        if epoch_error <= TARGET_ERROR:
            return model, history, epoch + 1, True
    return model, history, MAX_EPOCHS, False

def predict_real(model, points):
    return denormalize(model.forward(normalize(np.array(points, dtype=float))))

def closest_class(value):
    return C0 if abs(value - C0) < abs(value - C1) else C1

def calc_mae(model):
    pred = predict_real(model, X_raw)
    return float(np.mean(np.abs(pred - y_raw)))

def calc_accuracy(model):
    pred = predict_real(model, X_raw).flatten()
    correct = sum(closest_class(p) == t for p, t in zip(pred, y_raw.flatten()))
    return correct / len(X_raw) * 100

def hidden_state(model):
    z1 = np.dot(X_norm, model.W1) + model.b1
    active_counts = (z1 > 0).sum(axis=0)
    return z1, active_counts

def count_dead(model):
    _, active_counts = hidden_state(model)
    return int(np.sum(active_counts == 0))

CONFIGS = [('A', 'sigmoid', 'Конфигурация А (Сигмоида)', 'tab:blue'),
            ('B', 'relu', 'Конфигурация В (ReLU)', 'tab:red')]

results = {'A': [], 'B': []}
for name, act, _, _ in CONFIGS:
    for s in SEEDS:
        model, hist, epochs, conv = train_model(act, s)
        results[name].append({
            'seed': s, 'model': model, 'hist': hist, 'epochs': epochs, 'conv': conv,
            'es': hist[-1], 'acc': calc_accuracy(model), 'mae': calc_mae(model),
            'dead': count_dead(model)
        })

print("=== Результаты по 5 запускам ===")
for name, act, title, _ in CONFIGS:
    print(f"\n{title}")
    print(f"{'Seed':>6} | {'Сошлась':>8} | {'Эпох':>6} | {'Es':>9} | {'Accuracy':>9} | {'MAE':>8} | {'Мёртвых  
нейронов':>16}")
    print("-" * 82)
    for r in results[name]:
        print(f"{r['seed']:>6} | {'да' if r['conv'] else 'нет':>8} | {r['epochs']:>6} | {r['es']:>9.5f} | "
              f"{r['acc']:>8.1f}% | {r['mae']:>8.4f} | {r['dead']:>16}")

print("\n=== Сводная таблица сравнения ===")
header = f"{'Показатель':<42} | {'А (Сигмоида)':>14} | {'В (ReLU)':>14}"
print(header)
print("-" * len(header))

def stat_row(label, fa, fb):
    print(f"{label:<42} | {fa:>14} | {fb:>14}")

```

```

def collect(name):
    rs = results[name]
    ep_all = np.array([r['epochs'] for r in rs], dtype=float)
    ep_ok = np.array([r['epochs'] for r in rs if r['conv']], dtype=float)
    return {
        'n_conv': sum(r['conv'] for r in rs),
        'ep_all': ep_all,
        'ep_ok': ep_ok,
        'mae': np.array([r['mae'] for r in rs]),
        'acc': np.array([r['acc'] for r in rs]),
        'es': np.array([r['es'] for r in rs]),
        'dead': sum(r['dead'] for r in rs)
    }

stA, stB = collect('A'), collect('B')

def fmt_or_dash(arr, f):
    return f(arr) if len(arr) > 0 else "-"

stat_row("Сошлось запусков из 5", f"{stA['n_conv']}/5", f"{stB['n_conv']}/5")
stat_row("Эпох, среднее (все запуски)", f"{stA['ep_all'].mean():.1f}", f"{stB['ep_all'].mean():.1f}")
stat_row("Эпох, среднее (только сошедшиеся)", fmt_or_dash(stA['ep_ok'], lambda a: f"{a.mean():.1f}"),
        fmt_or_dash(stB['ep_ok'], lambda a: f"{a.mean():.1f}"))
stat_row("Эпох, медиана (все запуски)", f"{np.median(stA['ep_all']):.1f}", f"{np.median(stB['ep_all']):.1f}")
stat_row("Эпох, мин", f"{stA['ep_all'].min():.0f}", f"{stB['ep_all'].min():.0f}")
stat_row("Эпох, макс", f"{stA['ep_all'].max():.0f}", f"{stB['ep_all'].max():.0f}")
stat_row("Эпох, СКО (все запуски)", f"{stA['ep_all'].std():.1f}", f"{stB['ep_all'].std():.1f}")
stat_row("Итоговая Es, среднее", f"{stA['es'].mean():.5f}", f"{stB['es'].mean():.5f}")
stat_row("Accuracy, среднее", f"{stA['acc'].mean():.1f}%", f"{stB['acc'].mean():.1f}%")
stat_row("MAE, среднее", f"{stA['mae'].mean():.4f}", f"{stB['mae'].mean():.4f}")
stat_row("MAE, СКО", f"{stA['mae'].std():.4f}", f"{stB['mae'].std():.4f}")
stat_row("Мёртвых скрытых нейронов (сумма)", f"{stA['dead']}", f"{stB['dead']}")

# Выбираем представительную модель: первый успешный запуск (или просто первый)
rep = {}
for name in ('A', 'B'):
    successful = [r for r in results[name] if r['conv']]
    rep[name] = successful[0] if successful else results[name][0]

print(f"\n=== Представительные модели ===")
for name, act, title, _ in CONFIGS:
    r = rep[name]
    print(f"{title}: seed={r['seed']}, эпох = {r['epochs']}, Es = {r['es']:.5f}, "
        f"Accuracy = {r['acc']:.1f}%, MAE = {r['mae']:.4f}")

print("\n=== Скрытый слой представительной модели ReLU: преактивации z1 на 4 обучающих примерах ===")
z1_rep, act_counts = hidden_state(rep['B']['model'])
print("Вход (A, B) | z1 нейрона 1 | z1 нейрона 2")
for (a, b), z in zip(X_raw, z1_rep):
    print(f"({a:>4.0f}, {b:>4.0f}) | {z[0]:>12.4f} | {z[1]:>12.4f}")
print(f"Число примеров с z1 > 0 (нейрон активен): нейрон 1 = {act_counts[0]}, нейрон 2 = {act_counts[1]}")

print(f"\n=== Дополнительный эксперимент: {EXTRA_SEEDS} запусков с seed = 1..{EXTRA_SEEDS} ===")
extra = {}
for name, act, title, _ in CONFIGS:
    conv_flags, epochs_list, dead_list = [], [], []
    for s in range(1, EXTRA_SEEDS + 1):
        model, hist, epochs, conv = train_model(act, s)
        conv_flags.append(conv)
        epochs_list.append(epochs)
        dead_list.append(count_dead(model))
    conv_flags = np.array(conv_flags)
    epochs_arr = np.array(epochs_list, dtype=float)
    dead_arr = np.array(dead_list)
    extra[name] = (conv_flags, epochs_arr, dead_arr)
    ok = epochs_arr[conv_flags]
    mean_ok = f"{ok.mean():.1f}" if len(ok) else "-"
    med_ok = f"{np.median(ok):.1f}" if len(ok) else "-"
    print(f"{title}: сошлось {conv_flags.sum()}/{EXTRA_SEEDS}, среднее число эпох (сошедшиеся) = {mean_ok}, "
        f"медиана = {med_ok}, запусков с мёртвым нейроном = {int((dead_arr > 0).sum())}")
conv_B = extra['B'][0]
dead_B = extra['B'][2]
print(f"ReLU: среди запусков с мёртвым нейроном сошлось {int(conv_B[dead_B > 0].sum())} из {int((dead_B > 0).sum())}")
print(f"ReLU: среди запусков без мёртвых нейронов сошлось {int(conv_B[dead_B == 0].sum())} из {int((dead_B == 0).sum())}")

print("\nОткрывается первый график (закройте окно, чтобы появился следующий)...")
fig1 = plt.figure(figsize=(10, 6))
for name, act, title, color in CONFIGS:

```

```

hist = rep[name]['hist']
plt.plot(np.arange(1, len(hist) + 1), hist, label=f"{title}, seed={rep[name]['seed']}", color=color, linewidth=2)
plt.axhline(TARGET_ERROR, color='black', linestyle=':', linewidth=1.5, label=f'Попор Ее = {TARGET_ERROR}')
plt.yscale('log')
plt.title('Сходимость: суммарная ошибка Es от номера эпохи')
plt.xlabel('Эпоха')
plt.ylabel('Суммарная ошибка Es (ВСЕ, логарифмическая шкала)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
fig1.savefig('lab3_convergence.png', dpi=150)
plt.show()

print("Открывается второй график (закройте окно, чтобы появился следующий)...")
fig2, ax = plt.subplots(figsize=(10, 6))
x = np.arange(len(SEEDS))
width = 0.35
for offset, (name, act, title, color) in zip((-width / 2, width / 2), CONFIGS):
    for i, r in enumerate(results[name]):
        bar = ax.bar(x[i] + offset, r['epochs'], width, color=color, edgecolor='black',
                    alpha=0.85, hatch=None if r['conv'] else '///')
        ax.text(x[i] + offset, r['epochs'], str(r['epochs']), ha='center', va='bottom', fontsize=9)
ax.set_title('Число эпох по 5 запускам\n(штриховка = критерий останова не достигнут за MAX_EPOCHS)')
ax.set_xlabel('Запуск')
ax.set_ylabel('Эпохи')
ax.set_xticks(x)
ax.set_xticklabels([f'Run {i + 1}\n(seed={s})' for i, s in enumerate(SEEDS)])
legend_items = [Patch(facecolor=c, edgecolor='black', label=t) for _, _, t, c in CONFIGS]
legend_items.append(Patch(facecolor='white', edgecolor='black', hatch='///', label='Не сошлась'))
ax.legend(handles=legend_items)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
fig2.savefig('lab3_epochs.png', dpi=150)
plt.show()

print("Открывается третий график (закройте окно, чтобы появился следующий)...")
grid_a = np.linspace(-10, 10, 300)
xx, yy = np.meshgrid(grid_a, grid_a)
grid_points = np.c_[xx.ravel(), yy.ravel()]
fig3, axes = plt.subplots(1, 2, figsize=(14, 6))
cmap = plt.cm.RdYlBu
for ax, (name, act, title, color) in zip(axes, CONFIGS):
    Z = predict_real(rep[name]['model'], grid_points).reshape(xx.shape)
    cf = ax.contourf(xx, yy, Z, levels=50, cmap=cmap, vmin=C0, vmax=C1)
    ax.contour(xx, yy, Z, levels=[(C0 + C1) / 2], colors='black', linewidths=1.5, linestyles='--')
    ax.scatter(X_raw[:, 0], X_raw[:, 1], c=y_raw.flatten(), cmap=cmap, vmin=C0, vmax=C1,
              edgecolors='k', s=140, linewidths=1.5, zorder=3)
    ax.set_title(f'{title}\nseed={rep[name]["seed"]}, эпох={rep[name]["epochs"]}')
    ax.set_xlabel('Вход A')
    ax.set_ylabel('Вход B')
    ax.set_xlim(-10, 10)
    ax.set_ylim(-10, 10)
    fig3.colorbar(cf, ax=ax, label='Выход сети в шкале [c0; c1]')
plt.suptitle('Разделяющая поверхность: пунктир — линия порога (c0+c1)/2; точки — обучающая выборка',
            fontsize=12, fontweight='bold')
plt.tight_layout()
fig3.savefig('lab3_surfaces.png', dpi=150)
plt.show()

print("Открывается четвёртый график (закройте окно, чтобы появился следующий)...")
fig4, ax = plt.subplots(figsize=(8, 6))
labels = ['Конфигурация A\n(Сигмоида)', 'Конфигурация B\n(ReLU)']

mae_A_ok = [r['mae'] for r in results['A'] if r['conv']]
mae_B_ok = [r['mae'] for r in results['B'] if r['conv']]

means = [np.mean(mae_A_ok), np.mean(mae_B_ok) if mae_B_ok else 0]
stds = [np.std(mae_A_ok), np.std(mae_B_ok) if len(mae_B_ok) > 1 else 0]

bars = ax.bar(labels, means, yerr=stds, capsize=8, color=['tab:blue', 'tab:red'],
            width=0.6, edgecolor='black', alpha=0.85)
for bar, val in zip(bars, means):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01, f'{val:.4f}',
            ha='center', va='bottom', fontsize=11, fontweight='bold')
ax.set_title('Средняя абсолютная ошибка в шкале [c0; c1]\n(только успешные запуски ± СКО)')
ax.set_ylabel('MAE')
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
fig4.savefig('lab3_mae.png', dpi=150)
plt.show()

print("\n=== РЕЖИМ ФУНКЦИОНИРОВАНИЯ ===")
model_A = rep['A']['model']

```

```
model_B = rep['B']['model']

def describe(model, a, b):
    out_norm = float(model.forward(normalize(np.array([[a, b]]), dtype=float)))[0][0])
    out_real = float(denormalize(out_norm))
    return out_norm, out_real, closest_class(out_real)

test_cases = [(-9, -9), (-9, 6), (6, -9), (6, 6), (0, 0), (-5, 3), (2, -7)]
print(f"\n{'Вход (A, B)':<13} | {'A: ŷ(0;1)':>10} {'A: ŷ исх.':>10} {'A: класс':>9} || "
      f"{'B: ŷ(0;1)':>10} {'B: ŷ исх.':>10} {'B: класс':>9}")
print("-" * 85)
for a, b in test_cases:
    na, ra, ca = describe(model_A, a, b)
    nb, rb, cb = describe(model_B, a, b)
    print(f"{{a:>3}}, {{b:>3}} | {{na:>10.4f}} {{ra:>10.4f}} {{ca:>9.0f}} || {{nb:>10.4f}} {{rb:>10.4f}} {{cb:>9.0f}}")

print("\n--- Интерактивный режим (введите 'exit' для выхода) ---")
print("Введите A и B из диапазона [-10; 10] через пробел (например: -9 6)")
while True:
    try:
        user_input = input("\nВходные значения A и B: ")
        if user_input.strip().lower() == 'exit':
            print("Завершение работы.")
            break
        a, b = map(float, user_input.split())
        if not (-10 <= a <= 10 and -10 <= b <= 10):
            print("Значения должны лежать в диапазоне [-10; 10].")
            continue
        for label, model in (('Конфигурация A (Сигмоида)', model_A), ('Конфигурация B (ReLU)', model_B)):
            out_norm, out_real, cls = describe(model, a, b)
            print(f"{{label}}:")
            print(f"    Нормализованный выход : {{out_norm:.4f}}")
            print(f"    Выход в шкале [{{C0:.0f}}; {{C1:.0f}}] : {{out_real:.4f}}")
            print(f"    Ближайший класс : {{cls:.0f}}")
        except ValueError:
            print("Ошибка ввода. Введите два числа через пробел или 'exit'.")
```

Результат программы:

=== Результаты по 5 запускам ===

Конфигурация A (Сигмоида)							
Seed	Сошлась	Эпох	Es	Accuracy	MAE	Мёртвых нейронов	
10	да	703	0.04990	100.0%	0.1845	0	
42	да	756	0.04997	100.0%	0.1847	0	
123	да	671	0.04992	100.0%	0.1840	0	
777	да	647	0.04991	100.0%	0.1841	0	
2024	да	781	0.04991	100.0%	0.1840	0	

Конфигурация B (ReLU)							
Seed	Сошлась	Эпох	Es	Accuracy	MAE	Мёртвых нейронов	
10	да	110	0.04994	100.0%	0.1784	0	
42	да	134	0.04913	100.0%	0.1759	0	
123	да	88	0.04892	100.0%	0.1780	0	
777	нет	10000	1.52032	75.0%	3.7501	0	
2024	нет	10000	2.07872	75.0%	4.9902	0	

=== Сводная таблица сравнения ===

Показатель	A (Сигмоида)	B (ReLU)
Сошлось запусков из 5	5/5	3/5
Эпох, среднее (все запуски)	711.6	4066.4
Эпох, среднее (только сошедшиеся)	711.6	110.7
Эпох, медиана (все запуски)	703.0	134.0
Эпох, мин	647	88
Эпох, макс	781	10000

```

Эпох, СКО (все запуски) | 50.4 | 4844.8
Итоговая Es, среднее | 0.04992 | 0.74940
Аккурасу, среднее | 100.0% | 90.0%
MAE, среднее | 0.1842 | 1.8545
MAE, СКО | 0.0003 | 2.0911
Мёртвых скрытых нейронов (сумма) | 0 | 0

=== Представительные модели ===
Конфигурация A (Сигмоида): seed=10, эпох = 703, Es = 0.04990, Accuracy = 100.0%, MAE = 0.1845
Конфигурация B (ReLU): seed=10, эпох = 110, Es = 0.04994, Accuracy = 100.0%, MAE = 0.1784

=== Скрытый слой представительной модели ReLU: преактивации z1 на 4 обучающих примерах ===
Вход (A, B) | z1 нейрона 1 | z1 нейрона 2
( -9, -9) | -0.0069 | 2.3728
( -9, 6) | -4.0087 | -0.0273
( 6, -9) | 3.9975 | 4.9113
( 6, 6) | -0.0043 | 2.5111
Число примеров с z1 > 0 (нейрон активен): нейрон 1 = 1, нейрон 2 = 3

=== Дополнительный эксперимент: 100 запусков с seed = 1..100 ===
Конфигурация A (Сигмоида): сошлось 64/100, среднее число эпох (сошедшиеся) = 875.9, медиана = 736.5, запусков с мёртвым нейроном = 3
Конфигурация B (ReLU): сошлось 29/100, среднее число эпох (сошедшиеся) = 109.6, медиана = 107.0, запусков с мёртвым нейроном = 12
ReLU: среди запусков с мёртвым нейроном сошлось 0 из 12
ReLU: среди запусков без мёртвых нейронов сошлось 29 из 88

Открывается первый график (закройте окно, чтобы появился следующий)...
Открывается второй график (закройте окно, чтобы появился следующий)...
Открывается третий график (закройте окно, чтобы появился следующий)...
Открывается четвёртый график (закройте окно, чтобы появился следующий)...

```

=== РЕЖИМ ФУНКЦИОНИРОВАНИЯ ===

Вход (A, B)	A: $\hat{y}(0;1)$	A: $\hat{y}$ исх.	A: класс	B: $\hat{y}(0;1)$	B: $\hat{y}$ исх.	B: класс
(-9, -9)	0.0119	-8.8215	-9	0.0069	-8.8971	-9
(-9, 6)	0.9832	5.7481	6	0.9667	5.5006	6
(6, -9)	0.9897	5.8450	6	0.9968	5.9521	6
(6, 6)	0.0102	-8.8476	-9	0.0042	-8.9365	-9
(0, 0)	0.0108	-8.8384	-9	0.0051	-8.9230	-9
(-5, 3)	0.5193	-1.2101	6	0.3535	-3.6968	-9
(2, -7)	0.7003	1.5049	6	0.7989	2.9830	6

--- Интерактивный режим (введите 'exit' для выхода) ---

Введите A и B из диапазона [-10; 10] через пробел (например: -9 6)

Входные значения A и B: -9 6

Конфигурация A (Сигмоида):

Нормализованный выход : 0.9832

Выход в шкале [-9; 6] : 5.7481

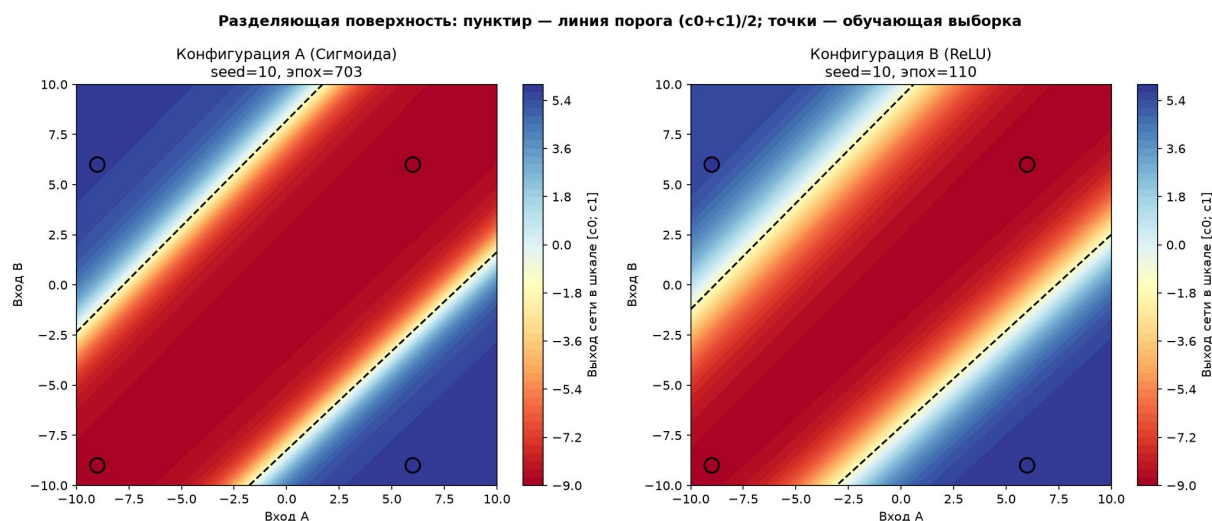
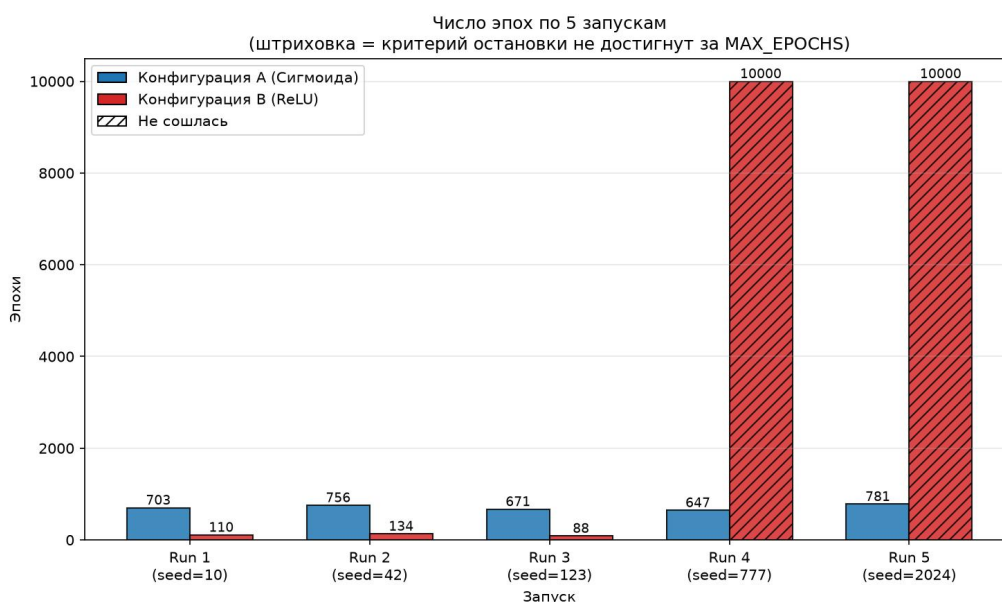
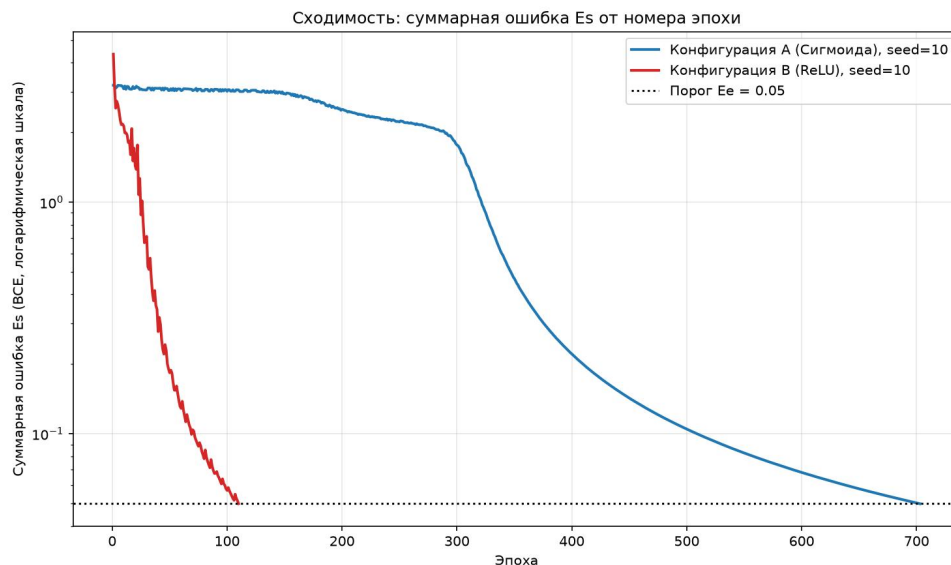
Ближайший класс : 6

Конфигурация B (ReLU):

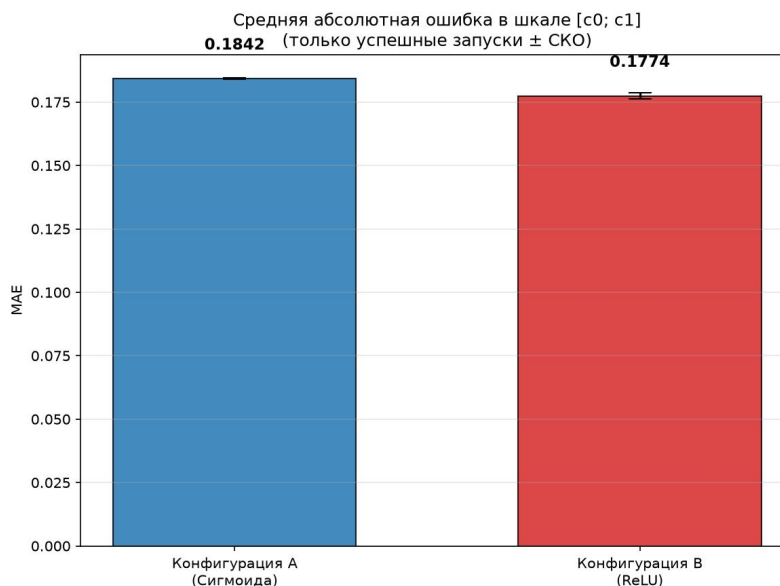
Нормализованный выход : 0.9667

Выход в шкале [-9; 6] : 5.5006

Ближайший класс : 6



Конфигурация A (Сигмоида) даёт гладкую нелинейную границу в форме «седла» с плавным переходом между классами. Конфигурация B (ReLU) формирует кусочно-линейную границу — переходы между классами резкие, угловатые, состоят из прямых отрезков. Это прямое следствие природы ReLU: $f(x) = \max(0, x)$ — кусочно-линейная функция, поэтому её композиция в сети тоже кусочно-линейна.



Вывод о проделанной работе:

1. Какая из активационных функций (Сигмоида или ReLU) обеспечила более быструю сходимость? Более устойчива?

ReLU быстрее: в успешных запусках — 110.7 эпох против 711.6 у Сигмоиды (в 6.4 раза). Сигмоида устойчивее: сошлась 5/5 запусков, ReLU — только 3/5 (seed=777 и 2024 не достигли порога за 10 000 эпох). На 100 запусках: Сигмоида — 64%, ReLU — 29%.

2. Как отличается форма разделяющей поверхности между конфигурациями А и В? Наблюдаются ли ожидаемые кусочно-линейные границы для ReLU?

Да, наблюдаются ожидаемые кусочно-линейные границы у ReLU — резкие угловатые переходы, в отличие от гладкой «седловой» формы у Сигмоиды. Это видно на heatmap: у ReLU цветовые зоны разделены прямыми линиями.

3. Почему ReLU часто предпочитают Сигмоиде в глубоких сетях? Подтверждается ли это на XOR?

ReLU избегают затухания градиента (производная = 1 при $x > 0$, у сигмоиды — максимум 0.25) и она вычислительно проще. На XOR это подтверждается: успешные запуски ReLU сходятся в 6.4 раза быстрее. Однако на малых сетях (2-2-1) проявляется проблема «мёртвых нейронов».

4. Изменилась ли способность сети решать XOR при переходе на ReLU, или оба варианта успешно справляются благодаря архитектуре?

Нет, не изменилась — обе конфигурации дают Accuracy = 100% на обучающей выборке. Способность решать XOR определяется архитектурой 2-2-1 (наличием скрытого слоя), а не конкретной активационной функцией. Обе функции обеспечивают необходимую нелинейность.

5. Есть ли случаи "мёртвых нейронов" при ReLU (нейронов, выдающих 0)? Как это повлияло на обучение?

Да, наблюдаются. В эксперименте на 100 запусках 12% запусков ReLU имели хотя бы один мёртвый нейрон, и ни один из них не сошёлся (0/12). Мёртвый нейрон ($z_1 \leq 0$ для всех примеров) имеет нулевую производную → веса не обновляются → сеть вырождается из 2-2-1 в 2-1-1 и не может решить XOR. В коде применена защита: Неинициализация + положительный bias ($b_1 = 1.0$).

6. Как выбор активационной функции соотносится с необходимостью многослойной архитектуры для XOR?

Не связана. Необходимость скрытого слоя определяется линейной неразделимостью XOR — никакая прямая линия не разделяет классы. Это свойство задачи, а не функции потерь или активации. И Сигмоида, и ReLU требуют архитектуру минимум 2-2-1; однослойный персептрон не решит XOR ни с одной из них.

7. Что показывает визуализация разделяющей поверхности для обеих конфигураций? Совпадает ли граница между классами с ожидаемой для XOR

Визуализация показывает, что обе конфигурации корректно решают XOR: точки (-9,-9) и (6,6) классифицируются как -9 (синяя область), (-9,6) и (6,-9) — как 6 (красная область). Пунктирная линия порога $(c_0 + c_1)/2 = -1.5$ чётко разделяет области. Форма границы — характерное для XOR «седло» из двух.